

Terraform in DevOps – Part 1

CSC-535 Cloud DevOps

Muhammad Tausif

Department of Computer Science
QAU

Lecture Roadmap

- 1 Recap: From Containers to Infrastructure
- 2 Terraform Architecture and Concepts
- 3 HCL: HashiCorp Configuration Language
- 4 Providers Deep Dive
- 5 Resources in Depth
- 6 Variables and Outputs in Depth
- 7 Terraform State Management
- 8 Data Sources
- 9 Terraform CLI Mastery
- 10 File Organization and Best Practices
- 11 Hands-On Walkthrough
- 12 Terraform in the DevOps Pipeline
- 13 Summary and Next Lecture Preview

Where We Are in the DevOps Journey

So far, we have covered:

- Containers and Docker (packaging applications)
- Kubernetes (orchestrating containers at scale)
- CI/CD pipelines (automating delivery)

But a critical question remains:

- **Who creates the infrastructure** where containers run?
- Who provisions VMs, networks, load balancers, databases?

The Missing Piece: Infrastructure Provisioning

Docker packages the application.
Kubernetes deploys the application.

But **before** any of that:

- Someone must create cloud servers.
- Someone must set up networking.
- Someone must configure security groups.
- Someone must provision storage.

Key Insight

Infrastructure must exist before deployment can happen.

Manual Infrastructure: The Old Way

Traditionally, teams provisioned infrastructure by:

- Logging into cloud console (AWS, Azure, GCP).
- Clicking through web interfaces.
- Running ad-hoc CLI commands.
- Writing custom shell scripts.

Problems:

- Not reproducible.
- No audit trail.
- Configuration drift over time.
- Hard to scale across environments.

Infrastructure as Code: The DevOps Way

Infrastructure as Code (IaC) means:

- Define infrastructure in text files.
- Store in version control (Git).
- Apply changes through automation.
- Treat infrastructure like software.

Core Principle

Infrastructure = Code = Versioned = Repeatable

IaC Tools Landscape

Several IaC tools exist:

Tool	Approach	Scope
Terraform	Declarative	Multi-cloud
CloudFormation	Declarative	AWS only
Pulumi	Imperative	Multi-cloud
Ansible	Procedural	Configuration
ARM Templates	Declarative	Azure only

Terraform is the most widely adopted multi-cloud IaC tool.

Why Terraform for This Course?

- Cloud-agnostic: works with AWS, Azure, GCP, and more.
- Large ecosystem of providers (4000+).
- Declarative syntax: describe *what* you want, not *how*.
- Strong community and industry adoption.
- Open-source with enterprise features available.

Industry Reality

Most DevOps job postings list Terraform as a required skill.

What is Terraform?

- Open-source tool created by HashiCorp (2014).
- Automates provisioning of infrastructure resources.
- Uses its own language: HCL (HashiCorp Configuration Language).
- Maintains a **state file** to track real-world resources.

One Sentence Definition

Terraform lets you write code that creates and manages cloud infrastructure.

Declarative vs Imperative

Imperative (How to do it):

- Step 1: Create VPC
- Step 2: Create subnet
- Step 3: Launch instance in subnet

Declarative (What you want):

- I want a VPC, a subnet, and an instance.
- Terraform figures out the order and dependencies.

Terraform is Declarative

You describe the desired end state. Terraform determines the steps.

Terraform Architecture Overview

Terraform has three main components:

① Terraform Core

- Reads configuration files.
- Builds resource dependency graph.
- Determines execution plan.

② Providers

- Plugins that interact with APIs (AWS, Azure, etc.).
- Translate HCL into API calls.

③ State

- Tracks what resources exist.
- Maps configuration to real-world resources.

How Terraform Works Internally

When you run `terraform apply`:

- 1 Terraform reads all `.tf` files in the directory.
- 2 Builds a **dependency graph** of resources.
- 3 Compares desired state (code) with current state (state file).
- 4 Generates an **execution plan** (what to create, update, delete).
- 5 Calls provider APIs to make changes.
- 6 Updates the state file.

Key Equation

$$Plan = Desired\ State - Current\ State$$

Terraform Dependency Graph

Terraform automatically determines resource ordering.

Example:

- VPC must exist before Subnet.
- Subnet must exist before EC2 Instance.
- Security Group must exist before EC2 Instance.

Terraform builds a Directed Acyclic Graph (DAG):

$$VPC \rightarrow Subnet \rightarrow EC2$$
$$VPC \rightarrow Security\ Group \rightarrow EC2$$

Independent resources are created **in parallel**.

Introduction to HCL

HCL is Terraform's configuration language.

- Human-readable syntax.
- Not a general-purpose programming language.
- Designed specifically for infrastructure definition.
- Files end with `.tf` extension.

Design Philosophy

Easy enough for operators to read, powerful enough for automation.

HCL Basic Syntax

HCL uses a block-based structure:

```
block_type "label1" "label2" {  
    argument1 = "value1"  
    argument2 = 42  
    argument3 = true  
}
```

- **Block type:** what kind of object (resource, variable, output).
- **Labels:** identifiers for the block.
- **Arguments:** key-value configuration inside the block.

HCL Data Types

Terraform supports these data types:

Type	Example
String	"hello"
Number	42, 3.14
Bool	true, false
List	["a", "b", "c"]
Map	{key = "value"}
Null	null

Complex types:

- `list(string)` – list of strings
- `map(number)` – map of numbers
- `object({...})` – structured object

String Interpolation

HCL supports embedding expressions inside strings:

```
variable "env" {  
    default = "production"  
}  
  
resource "aws_instance" "web" {  
    tags = {  
        Name = "server-${var.env}"  
    }  
}
```

Result: Tag Name = server-production

Syntax

Use `${...}` to embed variables and expressions inside strings.

Comments in HCL

Three styles of comments:

Single-line comment (most common)

// Also a single-line comment

```
/*  
    Multi-line  
    comment block  
*/
```

Best Practice

Use # for single-line comments. It is the community standard.

What are Providers?

Providers are **plugins** that let Terraform interact with external APIs.

- Each cloud platform has its own provider.
- Providers are downloaded during `terraform init`.
- They translate HCL into platform-specific API calls.

Analogy

Provider is like a **driver** for a database. Terraform is the application; the provider connects it to the cloud.

Provider Configuration

```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
}
```

- `required_providers`: declares which providers are needed.
- `provider`: configures the provider with settings.

Provider Versioning

Version constraints control which provider version is used:

Constraint	Meaning
<code>= 5.0</code>	Exactly version 5.0
<code>>= 5.0</code>	Version 5.0 or higher
<code>~> 5.0</code>	Any 5.x version (recommended)
<code>>= 5.0, < 6.0</code>	Between 5.0 and 6.0

Why Version Lock?

Provider updates can introduce breaking changes. Locking versions ensures stability.

Popular Terraform Providers

Provider	Purpose
aws	Amazon Web Services
azurerm	Microsoft Azure
google	Google Cloud Platform
kubernetes	Kubernetes clusters
docker	Docker containers
github	GitHub repositories
helm	Helm charts
local	Local files
random	Random values

Terraform Registry: `registry.terraform.io`

Multiple Providers

You can use multiple providers in one project:

```
provider "aws" {  
    region = "us-east-1"  
    alias   = "east"  
}
```

```
provider "aws" {  
    region = "eu-west-1"  
    alias   = "europe"  
}
```

```
resource "aws_instance" "us_server" {  
    provider = aws.east  
    # ...  
}
```

Use Case

Multi-region deployments or multi-cloud architectures.

What is a Resource?

A **resource** is the most important element in Terraform.

- Each resource block describes one infrastructure object.
- Examples: a VM, a database, a DNS record, a file.

Syntax Pattern

```
resource "TYPE" "NAME" { ... }
```

- **TYPE**: what kind of resource (e.g., `aws_instance`).
- **NAME**: local identifier (used only within Terraform).

Resource Example: AWS EC2

```
resource "aws_instance" "web_server" {  
    ami            = "ami-0c55b159cbfaffe1f0"  
    instance_type = "t2.micro"  
  
    tags = {  
        Name           = "WebServer"  
        Environment    = "production"  
    }  
}
```

- Creates one EC2 instance in AWS.
- Uses specified AMI and instance type.
- Adds metadata tags.

Resource Example: AWS S3 Bucket

```
resource "aws_s3_bucket" "data_bucket" {  
    bucket = "my-app-data-2024"  
  
    tags = {  
        Purpose = "Application Data"  
    }  
}
```

- Creates an S3 bucket for storing data.
- Bucket name must be globally unique.

Resource Example: Azure Resource Group

```
resource "azurerm_resource_group" "rg" {  
  name      = "my-app-resources"  
  location = "East US"  
}
```

- Creates a resource group in Azure.
- All Azure resources belong to a resource group.

Multi-Cloud Capability

Same Terraform workflow works for AWS, Azure, and GCP.

Resource Lifecycle

Every resource goes through a lifecycle:

- ① **Create**: Resource does not exist → Terraform creates it.
- ② **Read**: Terraform checks current state of the resource.
- ③ **Update**: Configuration changed → Terraform updates in-place.
- ④ **Delete**: Resource removed from code → Terraform destroys it.

This is called **CRUD** operations.

Important

Some updates require **destroy and recreate** (e.g., changing AMI).

Lifecycle Meta-Arguments

Control resource behavior with lifecycle blocks:

```
resource "aws_instance" "web" {  
  # ...  
  
  lifecycle {  
    create_before_destroy = true  
    prevent_destroy       = true  
    ignore_changes        = [tags]  
  }  
}
```

- `create_before_destroy`: new resource before old one is removed.
- `prevent_destroy`: blocks accidental deletion.
- `ignore_changes`: skip updates for specified attributes.

Resource Dependencies: Implicit

Terraform detects dependencies automatically:

```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
}  
  
resource "aws_subnet" "public" {  
  vpc_id      = aws_vpc.main.id  
  cidr_block  = "10.0.1.0/24"  
}
```

- Subnet references `aws_vpc.main.id`.
- Terraform knows: create VPC first, then Subnet.

Resource Dependencies: Explicit

When there is no attribute reference, use `depends_on`:

```
resource "aws_instance" "app" {  
  ami          = "ami-123456"  
  instance_type = "t2.micro"  
  
  depends_on = [aws_iam_role_policy.s3_access]  
}
```

When to Use

Only when Terraform cannot detect the dependency automatically. Prefer implicit dependencies.

Why Variables?

Hardcoding values is bad practice:

- Cannot reuse the same code for different environments.
- Secrets exposed in code.
- Changes require editing source files.

Variables solve this:

- Parameterize your configuration.
- Same code, different inputs.
- Separate configuration from logic.

Code + Variables = Flexible Infrastructure

Declaring Variables

```
variable "instance_type" {  
    description = "EC2 instance size"  
    type        = string  
    default     = "t2.micro"  
}
```

```
variable "instance_count" {  
    description = "Number of instances"  
    type        = number  
    default     = 1  
}
```

```
variable "enable_monitoring" {  
    description = "Enable detailed monitoring"  
    type        = bool  
    default     = false  
}
```

Variable Attributes

Each variable can have:

Attribute	Purpose
description	Human-readable explanation
type	Data type constraint
default	Value if none provided
validation	Custom validation rules
sensitive	Hide value from output
nullable	Allow null values

Best Practice

Always provide description and type for every variable.

Complex Variable Types

```
variable "availability_zones" {  
    type      = list(string)  
    default   = ["us-east-1a", "us-east-1b"]  
}
```

```
variable "instance_tags" {  
    type = map(string)  
    default = {  
        Environment = "dev"  
        Team         = "devops"  
    }  
}
```

- Lists are ordered collections.
- Maps are key-value pairs.

Variable Validation

Add custom rules to catch errors early:

```
variable "instance_type" {  
    type = string  
  
    validation {  
        condition = contains(  
            ["t2.micro", "t2.small", "t3.micro"],  
            var.instance_type  
        )  
        error_message = "Must be t2.micro,  
            t2.small, or t3.micro."  
    }  
}
```

Fail Fast

Validation catches mistakes before any cloud API call is made.

Ways to Set Variable Values

Variables can be set in multiple ways (in order of precedence):

- 1 Command line: `-var="key=value"`
- 2 Variable file: `terraform.tfvars`
- 3 Environment variable: `TF_VAR_key`
- 4 Default value in declaration.
- 5 Interactive prompt (if no default).

Common Pattern

Use `terraform.tfvars` for environment-specific values. Use defaults for sensible common values.

terraform.tfvars Example

File: terraform.tfvars

```
instance_type      = "t3.medium"  
instance_count     = 3  
enable_monitoring = true
```

Multiple var files for different environments:

```
terraform apply -var-file="prod.tfvars"  
terraform apply -var-file="staging.tfvars"
```

Environment Strategy

Same code, different .tfvars files per environment.

Sensitive Variables

Mark variables as sensitive to hide their values:

```
variable "db_password" {  
    description = "Database password"  
    type        = string  
    sensitive    = true  
}
```

- Value hidden in terraform plan output.
- Value hidden in terraform apply output.
- Still stored in state file (important security note).

Warning

Sensitive variables are **not encrypted** in state. Use remote state with encryption for production.

Output Values

Outputs expose information after terraform apply:

```
output "instance_ip" {  
    description = "Public IP of the server"  
    value       = aws_instance.web.public_ip  
}
```

```
output "bucket_arn" {  
    description = "ARN of the S3 bucket"  
    value       = aws_s3_bucket.data.arn  
}
```

Uses of outputs:

- Display important values to the user.
- Pass values between Terraform modules.
- Feed values into CI/CD pipelines.

Querying Outputs

After apply, query specific outputs:

```
terraform output  
terraform output instance_ip  
terraform output -json
```

- `terraform output`: shows all outputs.
- `terraform output instance_ip`: shows one output.
- `terraform output -json`: machine-readable format.

CI/CD Integration

Scripts can capture outputs:

```
IP=$(terraform output -raw instance_ip)
```

What is Terraform State?

State is Terraform's **memory** of what it has created.

- Stored in `terraform.tfstate` (JSON file).
- Maps configuration to real-world resources.
- Tracks resource IDs, attributes, and metadata.

Why State Exists

Cloud APIs do not know about your Terraform code. State bridges the gap between code and reality.

State: The Source of Truth

Terraform uses three sources of information:

- 1 **Configuration** (.tf files) – what you *want*.
- 2 **State** (.tfstate file) – what Terraform *thinks* exists.
- 3 **Real World** (cloud API) – what *actually* exists.

Terraform reconciles these three:

$$\text{Changes} = \text{Configuration} - \text{State}$$

Then verifies against real world during `refresh`.

Problems with Local State

By default, state is stored locally:

- Only one person can work at a time.
- No locking mechanism.
- Risk of data loss if file is deleted.
- Secrets stored in plain text on disk.

Local State is Fine For

Learning and personal projects only.

Remote State Backend

For teams, use **remote backends**:

```
terraform {  
  backend "s3" {  
    bucket          = "my-terraform-state"  
    key             = "prod/terraform.tfstate"  
    region          = "us-east-1"  
    encrypt         = true  
    dynamodb_table = "terraform-locks"  
  }  
}
```

Benefits:

- Shared access for team members.
- State locking (prevents concurrent changes).
- Encryption at rest.
- Versioning and backup.

Common Remote Backends

Backend	Provider
S3 + DynamoDB	AWS
Azure Blob Storage	Azure
Google Cloud Storage	GCP
Terraform Cloud	HashiCorp
Consul	HashiCorp
PostgreSQL	Self-hosted

Recommendation

Use S3 backend for AWS projects. Use Terraform Cloud for multi-cloud teams.

State Locking

State locking prevents concurrent modifications:

- Developer A runs `terraform apply`.
- Lock is acquired on state file.
- Developer B tries to run `terraform apply`.
- Developer B is blocked until A finishes.

Without Locking

Two people applying at the same time can corrupt the state file and create duplicate resources.

State Commands

Useful commands for state management:

```
terraform state list
terraform state show aws_instance.web
terraform state mv aws_instance.old aws_instance.new
terraform state rm aws_instance.web
terraform state pull
```

- `list`: show all tracked resources.
- `show`: inspect a specific resource.
- `mv`: rename a resource in state.
- `rm`: remove resource from state (does not destroy it).
- `pull`: download remote state locally.

What are Data Sources?

Data sources let you **read** information from existing infrastructure.

- Resources **create** things.
- Data sources **query** things that already exist.

Use cases:

- Look up an existing VPC ID.
- Find the latest AMI for an EC2 instance.
- Read information from another Terraform state.

Key Difference

resource = write operation. data = read operation.

Data Source Example: Latest AMI

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu--amd64-server-*"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
}
```

Benefit

Always uses the latest Ubuntu AMI without manual updates.

Data Source Example: Existing VPC

```
data "aws_vpc" "existing" {
  filter {
    name     = "tag:Name"
    values   = ["production-vpc"]
  }
}

resource "aws_subnet" "new_subnet" {
  vpc_id      = data.aws_vpc.existing.id
  cidr_block  = "10.0.5.0/24"
}
```

- Looks up a VPC that already exists in AWS.
- Uses its ID to create a new subnet.
- No risk of modifying the existing VPC.

Core Terraform Commands

Every Terraform workflow uses these commands:

Command	Purpose
<code>terraform init</code>	Initialize, download providers
<code>terraform plan</code>	Preview changes
<code>terraform apply</code>	Execute changes
<code>terraform destroy</code>	Remove all resources
<code>terraform fmt</code>	Format code consistently
<code>terraform validate</code>	Check syntax errors

Workflow Order

`init` → `validate` → `plan` → `apply`

terraform init

```
terraform init
```

What happens during init:

- Downloads provider plugins.
- Initializes backend (state storage).
- Downloads modules.
- Creates `.terraform/` directory.
- Creates `.terraform.lock.hcl` (dependency lock).

Must Run

Run init when: starting a new project, adding a provider, changing backend, or after cloning a repo.

terraform plan

```
terraform plan
terraform plan -out=tfplan
```

Plan output shows:

- + Resources to be created.
- - Resources to be destroyed.
- ~ Resources to be modified.
- -/+ Resources to be replaced.

Safety Net

Always review the plan before applying. Never apply blindly.

terraform apply

```
terraform apply
terraform apply tfplan
terraform apply -auto-approve
```

- Default: shows plan and asks for confirmation.
- `apply tfplan`: applies a saved plan (no re-prompt).
- `-auto-approve`: skip confirmation (use in CI/CD only).

Production Best Practice

```
plan -out=tfplan → review → apply tfplan
```

terraform destroy

```
terraform destroy
terraform destroy -target=aws_instance.web
```

- Removes all resources managed by the current state.
- `-target`: destroy only specific resources.
- Always asks for confirmation.

Caution

In production, `terraform destroy` can delete your entire infrastructure. Use with extreme care.

terraform fmt and validate

```
terraform fmt
terraform fmt -recursive
terraform validate
```

fmt:

- Automatically formats .tf files.
- Ensures consistent style across team.

validate:

- Checks syntax and internal consistency.
- Does not access any cloud API.
- Catches errors before plan.

Additional Useful Commands

```
terraform show
terraform graph
terraform import aws_instance.web i-12345
terraform workspace list
terraform refresh
```

- show: display current state in readable format.
- graph: generate dependency graph (DOT format).
- import: bring existing resources under Terraform management.
- workspace: manage multiple state files.
- refresh: sync state with real world.

Terraform Project File Structure

A well-organized Terraform project:

File	Purpose
<code>main.tf</code>	Primary resource definitions
<code>variables.tf</code>	Variable declarations
<code>outputs.tf</code>	Output definitions
<code>providers.tf</code>	Provider configuration
<code>terraform.tfvars</code>	Variable values
<code>backend.tf</code>	State backend config
<code>versions.tf</code>	Version constraints

Convention

Terraform loads all `.tf` files in a directory. File names are for human readability.

Best Practices Summary

- 1 Always use version control (Git).
- 2 Use remote state with locking.
- 3 Never commit `terraform.tfstate` to Git.
- 4 Never commit secrets or `.tfvars` with passwords.
- 5 Use `terraform fmt` before commits.
- 6 Use `terraform validate` in CI pipelines.
- 7 Pin provider versions.
- 8 Use meaningful resource names.
- 9 Add descriptions to all variables and outputs.
- 10 Review `terraform plan` before every apply.

.gitignore for Terraform

Always include a .gitignore:

```
# Terraform
.terraform/
*.tfstate
*.tfstate.backup
*.tfvars
crash.log
.terraform.lock.hcl
```

Security

State files contain secrets. Variable files may contain passwords. Never push these to Git.

Demo: Complete Local Example

Step 1: Create main.tf

```
terraform {  
  required_providers {  
    local = {  
      source  = "hashicorp/local"  
      version = "~> 2.0"  
    }  
  }  
}  
  
resource "local_file" "greeting" {  
  filename = "${path.module}/hello.txt"  
  content  = "Hello from Terraform!"  
}
```

Demo: Running the Workflow

Step 2: Execute the workflow

```
$ terraform init
$ terraform plan
$ terraform apply
$ cat hello.txt
Hello from Terraform!
$ terraform destroy
```

Observe

Terraform created a file, tracked it in state, and cleaned it up on destroy.

Demo: Adding Variables

Enhanced version with variables:

```
variable "message" {  
    description = "Content of the file"  
    type        = string  
    default     = "Hello from Terraform!"  
}  
  
resource "local_file" "greeting" {  
    filename = "${path.module}/hello.txt"  
    content  = var.message  
}  
  
output "file_path" {  
    value = local_file.greeting.filename  
}
```

Demo: Overriding Variables

```
$ terraform apply -var="message=Custom Message"
```

```
$ terraform apply \  
    -var-file="custom.tfvars"
```

```
$ TF_VAR_message="From ENV" terraform apply
```

Key Takeaway

Same code, different behavior based on input variables. This is the power of parameterized infrastructure.

Where Terraform Fits in CI/CD

In a complete DevOps pipeline:

- 1 Developer pushes code to Git.
- 2 CI server (Jenkins/GitHub Actions) triggers.
- 3 **Terraform provisions infrastructure.**
- 4 Ansible/scripts configure servers.
- 5 Docker image is built and pushed.
- 6 Kubernetes deploys the application.
- 7 Monitoring tools begin tracking.

Terraform's Role

Terraform runs **early** in the pipeline – creating the platform on which everything else runs.

Terraform + Docker + Kubernetes

How the tools work together:

- **Terraform** creates:
 - EKS/AKS/GKE cluster
 - VPC and networking
 - IAM roles and policies
 - Container registry
- **Docker** packages the application into images.
- **Kubernetes** deploys and manages the containers.

Terraform → Docker → Kubernetes

(Infrastructure) → (Packaging) → (Deployment)

Infrastructure Lifecycle Management

Terraform enables full lifecycle management:

- ❶ **Provision:** Create infrastructure from code.
- ❷ **Update:** Modify infrastructure incrementally.
- ❸ **Scale:** Add more resources as needed.
- ❹ **Audit:** Review changes through Git history.
- ❺ **Decommission:** Destroy infrastructure cleanly.

DevOps Principle

Infrastructure has a lifecycle just like software. Terraform manages it with the same rigor.

Lecture 7 Summary

Today we covered:

- Why Infrastructure as Code is essential in DevOps.
- Terraform architecture: Core, Providers, State.
- HCL syntax: blocks, types, interpolation.
- Providers: configuration, versioning, multi-provider setups.
- Resources: CRUD lifecycle, dependencies, lifecycle rules.
- Variables: types, validation, sensitivity, precedence.
- Outputs: exposing values and CI/CD integration.
- State: local vs remote, locking, state commands.
- Data sources: reading existing infrastructure.
- CLI commands: init, plan, apply, destroy, fmt, validate.

Next Lecture: Terraform Part 2

In the next lecture, we will cover:

- Terraform Modules (reusable infrastructure components).
- Workspaces (managing multiple environments).
- Expressions and Functions (conditionals, loops, built-ins).
- Provisioners (running scripts on resources).
- Terraform Cloud and Team Workflows.
- Real-world AWS infrastructure project.
- Terraform with Kubernetes provider.
- Testing and CI/CD integration patterns.

Preparation

Install Terraform on your machine and try the local file example before next class.